Answer, Search, or Abstain: Can RAG Agents Tell When Their Context Is Enough?

Abstract

Retrieval augmented agents decide on their own when to stop searching and answer. When that decision goes wrong, they answer confidently from passages that look relevant but do not contain the answer, or they stop after the first hop of a multi-hop question. We study this decision directly. We build a benchmark from HotpotQA with three controlled conditions (answerable in one retrieval, answerable only after a second search, and unanswerable because the gold passages are removed) and derive the correct action at every agent step from which gold passages have been retrieved so far. Using this oracle, we compare five controllers that choose between answering, searching again, and abstaining: single shot RAG, a ReAct style prompt, a prompt with an explicit sufficiency checklist, a logistic regression over cheap retrieval signals, and a LoRA adapter trained on oracle labels. We evaluate open models from 1.5B to 7B parameters on a single Colab class GPU, with a GPT class model as a reference. We report decision accuracy, hallucination rate on unanswerable questions, answer quality, and cost. Our hypothesis is that prompting and scale do little to fix the failure, while a small trained decision policy closes most of the gap at low cost.

1. Introduction

RAG systems are now the default way to ground language models in documents, and agentic RAG lets the model search repeatedly before it answers. This adds a decision that plain RAG never had to make: is what I have retrieved enough? A retriever always returns its top k passages, even when none of them contain the answer, so the model never receives a signal that the evidence is missing. It has to judge sufficiency on its own.

In practice this judgment fails in two ways. On unanswerable questions, agents answer anyway, because a passage that is on topic looks like a passage that is useful. On multi-hop questions, agents stop after finding the bridge entity and answer from half the evidence. Both failures produce fluent, confident, wrong answers, which is the hardest kind of error to catch in deployment.

We ask one question: can a RAG agent learn to tell when its retrieved context is sufficient, and which approach to that decision gives the best tradeoff between accuracy and cost?

Our contributions are:

1. A three condition benchmark built from HotpotQA, where every agent step has an oracle label for the correct action (answer, search again, or abstain), derived from which gold passages have been retrieved.
2. A controlled comparison of five decision controllers, holding the retriever, reranker, and answer generator fixed so that differences come only from the decision.
3. A failure analysis using per condition confusion matrices, showing which wrong action each controller takes and where.
4. Ablations over model size, quantization, retrieval method, and step budget, all within a single Colab class GPU.

2. Related Work

Retrieval augmented and agentic generation. RAG [Lewis et al., 2020] conditions generation on retrieved passages. ReAct [Yao et al., 2023] interleaves reasoning with tool calls, and IRCoT [Trivedi et al., 2023] and FLARE [Jiang et al., 2023] retrieve repeatedly during multi-hop reasoning. These methods improve answers when evidence exists, but they are rarely evaluated on whether the agent stops at the right time or abstains when evidence is missing.

Learning when to retrieve. Self-RAG [Asai et al., 2024] trains a model to emit reflection tokens that decide when to retrieve and whether passages are relevant, with labels distilled from a stronger critic model. Our LoRA controller is closest to this idea. The difference is that our labels come from an exact oracle over gold passages, and our benchmark separates the three situations so that each kind of mistake can be measured on its own.

Abstention and unanswerable questions. SQuAD 2.0 [Rajpurkar et al., 2018] introduced unanswerable questions for reading comprehension. Selective prediction [Geifman and El-Yaniv, 2017; Kamath et al., 2020] studies when a model should decline to answer. The RGB benchmark [Chen et al., 2024] tests negative rejection in RAG, and Kadavath et al. [2022] show that models carry some signal about their own correctness. Most directly, recent work on sufficient context [Joren et al., 2025] argues that RAG errors should be analyzed by whether the context is sufficient, not just relevant. We build on that lens and turn it into a sequential decision inside an agent loop.

3. Demonstrating the Failure

Before proposing any fix, we show that the failure is real and common, including for a GPT class model. We run the C0 and C1 baselines and a GPT class model on a sample of each condition and collect cases like the following. (Placeholder: replace with real examples from our runs.)

Example A, unanswerable. Question: which year was the director of a given film born? The gold passages about the director were removed. The retriever returns passages about the film's cast and release. The agent answers with a birth year taken from a cast member's passage. The passages were relevant to the film but did not contain the fact asked for.

Example B, premature answer. A two hop question needs the bridge entity from passage one and the final fact from passage two. The first retrieval returns only passage one. The agent identifies the bridge entity correctly, then guesses the final fact instead of searching for it.

We group failures into four types and report how often each occurs:

Failure type	What happens	Condition where it appears	Frequency
Hallucinated answer	Answers when the gold passages are absent	Unanswerable	TBD
Premature answer	Answers before the second hop is retrieved	Multi-hop	TBD
Endless search	Keeps searching until the step limit when it should answer or abstain	Easy, Unanswerable	TBD
Unproductive rewrite	Search again with a query nearly identical to the last one	Multi-hop	TBD

4. Method

4.1 Benchmark

We use HotpotQA [Yang et al., 2018], where each question comes with the two supporting paragraphs needed to answer it. The corpus is the union of supporting and distractor paragraphs for our selected questions, roughly 50k to 100k paragraphs in one FAISS index. From the dev set we build 900 evaluation questions, 300 per condition:

- Easy: a single retrieval returns both supporting paragraphs.
- Multi-hop: the first retrieval misses the second hop paragraph, so the agent must search again.
- Unanswerable: the supporting paragraphs are filtered out at retrieval time, so no number of searches can find them.

Filtering at query time means we keep a single index and can switch a question between conditions without rebuilding anything.

4.2 Oracle labels

At step t, let G be the set of gold paragraphs for the question and R_t the set of paragraphs retrieved so far. The correct action is:

```
a^*_t = \begin{cases} \text{ANSWER} & \text{if } G \subseteq R_t \\
\text{SEARCH} & \text{if } G \not\subseteq R_t \text{ and } G \subseteq
\text{corpus} \\ \text{ABSTAIN} & \text{if } G \not\subseteq \text{corpus}
\end{cases}
```

This gives exact ground truth for every decision, not only for the final answer. It also gives free training labels for the learned controllers.

4.3 System

The agent runs a loop with at most three retrieval steps. Each step retrieves the top 20 passages with BGE embeddings, reranks them to the top 5 with a BGE cross encoder reranker, and passes them to the controller. The controller returns one of three actions. Answer sends the context to a fixed answer generator. Search again produces a rewritten query and loops back to retrieval. Abstain returns a fixed "not found" response. The retriever, reranker, and answer prompt are identical across all controllers.

4.4 Controllers

Each controller adds one ingredient to the one before it, so the gap between adjacent rows isolates what that ingredient is worth.

ID	Controller	How it decides	What it tests
C0	Single shot RAG	Always retrieves once and answers	Baseline floor for accuracy and cost
C1	ReAct prompt	Model reasons briefly, then picks an action; constrained decoding forces a valid action	Can the model judge sufficiency from instructions alone?
C2	Sufficiency checklist	Model lists the facts it needs and marks each found or missing with a cited passage; a rule maps the checklist to an action	Does structured reasoning fix the judgment?
C3	Signal classifier	Multiclass logistic regression on reranker scores, top two score sum, novelty of new passages, step number, and draft answer confidence	Can cheap signals replace the LLM's judgment?
C4	LoRA policy	3B model with a LoRA adapter, trained on oracle labels to output one action token	Does a small trained policy fix it?

C3 and C4 are trained on the same data: about 3k HotpotQA training questions run through the agent, with every step labeled by the oracle, giving roughly 5k to 8k examples. Train and evaluation questions never overlap.

5. Experimental Setup

Models and compute. The main model is Qwen2.5 3B Instruct (Llama 3.2 3B as a backup), served with vLLM on one Colab class GPU (T4 or L4). For the size ablation we also run 1.5B and 7B models, with the 7B model in 4 bit. A GPT class model runs the C1 prompt on 100 questions per condition through its API as a reference. Embeddings and reranking use BGE small and the BGE reranker.

Training. The C4 adapter uses LoRA on the 3B model in 4 bit, trained with cross entropy on the single action token only. The three actions are not equally common in the training data, so we balance classes by resampling. C3 is a scikit-learn logistic regression on the same labeled steps. We hold out 10% of the training questions for validation.

Metrics.

Group	Metric	Definition
Decision	Action accuracy	Share of steps where the chosen action matches the oracle, per condition
Decision	Confusion matrix	Predicted action vs oracle action, per condition
Outcome	Hallucination rate	Share of unanswerable questions that receive an answer
Outcome	EM and F1	Answer quality on easy and multi-hop questions
Cost	Retrieval calls	Average searches per question
Cost	Tokens and latency	Generated tokens and wall clock seconds per question

Ablations. Step budget (1, 2, 3), model size (1.5B, 3B, 7B), precision (4 bit vs 16 bit for the 3B model), and retriever (BM25 vs dense). Each ablation changes one factor and keeps the rest at the main setting.

Reproducibility. Fixed seeds, greedy decoding for all decisions, and all prompts and configs released with the code.

6. Results

The tables below are templates to fill from our runs. The sentences under each table state what we expect to find, so the report can be checked against them once the numbers are in.

6.1 Main comparison (3B model)

Controller	Action acc. Easy (%)	Action acc. Multi-hop (%)	Action acc. Unanswerable (%)	Hallucination rate (%)	F1	Retrieval calls	Latency (s)
C0 Single shot	TBD	TBD	TBD	TBD	TBD	1.0	TBD
C1 ReAct prompt	TBD	TBD	TBD	TBD	TBD	TBD	TBD
C2 Checklist	TBD	TBD	TBD	TBD	TBD	TBD	TBD
C3 Signal classifier	TBD	TBD	TBD	TBD	TBD	TBD	TBD
C4 LoRA policy	TBD	TBD	TBD	TBD	TBD	TBD	TBD
GPT reference (C1)	TBD	TBD	TBD	TBD	TBD	TBD	TBD

Expected: C0 hallucinates on nearly all unanswerable questions by design. C1 improves little over C0 because the model rarely chooses to abstain. C2 helps on multi-hop questions but costs more tokens. C4 gives the best action accuracy on unanswerable questions at close to C1's cost.

6.2 Confusion matrices

One 3x3 matrix per controller and condition, predicted action against oracle action. We show C1 and C4 in the main text and the rest in the appendix.

6.3 Ablations

Ablation	Setting	Action acc. (%)	Hallucination rate (%)	Latency (s)
Model size (C1)	1.5B / 3B / 7B	TBD	TBD	TBD
Precision (C4)	4 bit / 16 bit	TBD	TBD	TBD
Retriever (C4)	BM25 / dense	TBD	TBD	TBD
Step budget (C4)	1 / 2 / 3	TBD	TBD	TBD

Expected: larger models reduce hallucination only modestly, which supports the claim that scale alone does not fix the decision. Quantization costs little decision accuracy.

7. Analysis: Why Agents Misjudge Sufficiency

Our working explanation is that the model confuses relevance with sufficiency. Retrieved passages are almost always on topic, because that is what retrievers optimize for. Instruction tuned models are also trained to be helpful, so an on topic context plus a question pulls strongly toward producing an answer. Nothing in the input marks the difference between a passage about the right film and a passage containing the right fact.

We test this explanation three ways:

1. If the model judges by relevance, hallucination should rise with the reranker score of the distractors. We bin unanswerable questions by top distractor score and plot hallucination rate per bin.
2. If C3 matches C4, the decision is mostly predictable from retrieval signals, and the LLM adds little. If C4 clearly beats C3, the model is reading content that scores cannot capture.
3. If C2's checklists fail, we inspect whether the model marks a missing fact as found. That would show it cannot verify a fact against a passage, not just that it fails to ask.

8. Limitations and Future Work

All results come from one dataset. HotpotQA questions follow a regular two hop structure, and a trained controller may learn that structure rather than a general sense of sufficiency. Testing the C4 policy on a different QA dataset without retraining would show whether it transfers.

Our unanswerable condition is synthetic. Removing gold passages creates a clean signal, but real unanswerable questions often have partial or conflicting evidence instead of none.

The action space is small on purpose. A natural extension adds a fourth action, asking the user a clarifying question, evaluated on ambiguous questions from AmbigQA.

Finally, model sizes are capped by a single GPU, so we cannot say whether the failure persists at frontier scale beyond the GPT reference sample.

9. Conclusion

We asked whether a RAG agent can tell when its context is enough. We built a benchmark where the correct action is known at every step, and used it to compare five ways of making that decision. We expect to show that the failure is common, that prompting and scale alone do not fix it, and that a small policy trained on oracle labels does most of the work at low cost. Whatever the outcome, the per step oracle and confusion matrices give a direct way to measure a decision that most RAG evaluations never look at.

References

Citations below were written from memory and have not been checked against the papers. Verify titles, venues, and years before submission.

- Asai, A., et al. Self-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection. ICLR 2024.
- Chen, J., et al. Benchmarking Large Language Models in Retrieval-Augmented Generation. AAAI 2024.
- Geifman, Y., and El-Yaniv, R. Selective Classification for Deep Neural Networks. NeurIPS 2017.
- Hu, E., et al. LoRA: Low-Rank Adaptation of Large Language Models. ICLR 2022.
- Jiang, Z., et al. Active Retrieval Augmented Generation (FLARE). EMNLP 2023.
- Joren, H., et al. Sufficient Context: A New Lens on Retrieval Augmented Generation Systems. ICLR 2025.
- Kadavath, S., et al. Language Models (Mostly) Know What They Know. arXiv 2022.
- Kamath, A., Jia, R., and Liang, P. Selective Question Answering under Domain Shift. ACL 2020.
- Lewis, P., et al. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. NeurIPS 2020.
- Min, S., et al. AmbigQA: Answering Ambiguous Open-domain Questions. EMNLP 2020.
- Rajpurkar, P., Jia, R., and Liang, P. Know What You Don't Know: Unanswerable Questions for SQuAD. ACL 2018.
- Trivedi, H., et al. Interleaving Retrieval with Chain-of-Thought Reasoning for Knowledge-Intensive Multi-Step Questions (IRCoT). ACL 2023.
- Xiao, S., et al. C-Pack: Packaged Resources to Advance General Chinese Embedding (BGE). arXiv 2023.

- Yang, Z., et al. HotpotQA: A Dataset for Diverse, Explainable Multi-hop Question Answering. EMNLP 2018.
- Yao, S., et al. ReAct: Synergizing Reasoning and Acting in Language Models. ICLR 2023.